

SchoolParentMonorepo



✨ Your new, shiny [Nx workspace](#) is almost ready ✨ .

[Learn more about this workspace setup and its capabilities](#) or run `npx nx graph` to visually explore what was created. Now, let's get you up to speed!

Finish your CI setup

[Click here to finish setting up your workspace!](#)

Generate a library

```
npx nx g @nx/js:lib packages/pkg1 --publishable --importPath=@my-org/pkg1
```

Run tasks

To build the library use:

```
npx nx build pkg1
```

To run any task with Nx use:

```
npx nx <target> <project-name>
```

These targets are either [inferred automatically](#) or defined in the `project.json` or `package.json` files.

[More about running tasks in the docs »](#)

Versioning and releasing

To version and release the library use

```
npx nx release
```

Pass `--dry-run` to see what would happen without actually releasing the library.

[Learn more about Nx release »](#)

Keep TypeScript project references up to date

Nx automatically updates TypeScript [project references](#) in `tsconfig.json` files to ensure they remain accurate based on your project dependencies (`import` or `require` statements). This sync is automatically done when running tasks such as `build` or `typecheck`, which require updated references to function correctly.

To manually trigger the process to sync the project graph dependencies information to the TypeScript project references, run the following command:

```
npx nx sync
```

You can enforce that the TypeScript project references are always in the correct state when running in CI by adding a step to your CI job configuration that runs the following command:

```
npx nx sync:check
```

[Learn more about nx sync](#)

[Learn more about Nx on CI](#)

Install Nx Console

Nx Console is an editor extension that enriches your developer experience. It lets you run tasks, generate code, and improves code autocompletion in your IDE. It is available for VSCode and IntelliJ.

[Install Nx Console »](#)

Useful links

Learn more:

- [Learn more about this workspace setup](#)
- [Learn about Nx on CI](#)
- [Releasing Packages with Nx release](#)
- [What are Nx plugins?](#)

And join the Nx community:

- [Discord](#)
- [Follow us on X](#) or [LinkedIn](#)
- [Our Youtube channel](#)
- [Our blog](#)

Start the database

```
npm run docker:up
```

Stop the database

```
npm run docker:down
```

View database logs

```
npm run docker:logs
```

Reset database (if needed)

```
npm run docker:down
```

```
docker volume rm school-parent_postgres_data
```

```
npm run docker:up
```

```
npm run db:migrate
```

local db details:

Host: localhost

Port: 5432

Database: school_parent

Username: school_parent

Password: school_parent_password

Connection URL: postgresql://school_parent:school_parent_password@localhost:5432/school_parent

Connect to database

```
docker exec -it school_parent_db psql -U school_parent -d school_parent
```

```
\dt -- list tables
```

```
SELECT * FROM "TestItem";
```

```
\q
```

```
\l -- list databases
```

```
\dt -- list tables
```

```
\d table -- describe table
```

```
\q -- quit
```